AD23431 - STATISTICAL ANALYSIS AND COMPUTING

EXP 1: Implementation of Vectors, Matrices, and Arrays in R

I Vector Operations in R

"The Daily Grind" Sales Analysis

Analyze the sales performance of a local café over six months. You have data for two main product categories: **Coffee** and **Pastries**.

Using Vectors in R do the following:

a. Create the datasets representing monthly revenue (in USD) from January to June.
b. Calculate the total revenue for the cafe each month (Coffee + Pastries) and the overall total for the half-year.
c. Find the average monthly sales for coffee and determine the standard deviation to see how much sales fluctuate.
d. The owner wants to know which months the coffee sales exceeded **$3,000**. Use logical indexing to extract these months.
e. Calculate the difference in sales between June and January to see the growth over the period.

PROGRAM:

```
months <- c("January","February","March","April","May","June")
coffee_sales <- c(2400,2700,2950,3100,3250,3600)
pastry_sales <- c(1000,1150,1250,1400,1500,1650)
monthly_total <- coffee_sales + pastry_sales
grand_total <- sum(monthly_total)
names(monthly_total) <- months
print("Monthly Totals:")
print(monthly_total)
print(paste0("Grand Total Revenue: $",grand_total))
avg_coffee <- mean(coffee_sales)
sd_coffee <- sd(coffee_sales)
```

```
print(paste0("Average Coffee Sales: $",round(avg_coffee,2)))
print(paste0("Standard Deviation of Coffee Sales: $",round(sd_coffee,2)))
high_sales_index <- coffee_sales > 3000
high_sales_months <- months[high_sales_index]
print("Months with Coffee sales exceeding $3,000:")
print(high_sales_months)
growth <- monthly_total[6] - monthly_total[1]
print(paste0("Revenue Growth (June - Jan): $",growth))
```

OUTPUT:

```
[1] "Monthly Totals:"
 January February    March    April      May     June
    3400     3850     4200     4500     4750     5250
[1] "Grand Total Revenue: $25950"
[1] "Average Coffee Sales: $3000"
[1] "Standard Deviation of Coffee Sales: $420.71"
[1] "Months with Coffee sales exceeding $3,000:"
[1] "April" "May"   "June"
[1] "Revenue Growth (June - Jan): $1850"
```

II Vector Operations in R

"FitLife Gym" Membership Revenue Analysis

Analyze the monthly membership revenue of a fitness center for six months.

Using Vectors in R, do the following:

a. Create vectors representing monthly revenue (in USD) from **Basic** and **Premium** memberships (January–June).
b. Calculate the total revenue for each month and the overall revenue for the six-month period.
c. Find the average monthly revenue for Premium memberships and compute the standard deviation.
d. Identify the months in which Premium membership revenue exceeded **$5,000** using logical indexing.
e. Calculate the revenue growth from January to June for Basic memberships.

PROGRAM:

```
months <- c("January","February","March","April","May","June")
basic_revenue <- c(2800,3050,3350,3600,3950,4250)
premium_revenue <- c(4800,5100,5350,5600,5900,6250)
cat("Basic Revenue:",basic_revenue,"\n")
cat("Premium Revenue:",premium_revenue,"\n\n")
total_monthly_revenue <- basic_revenue + premium_revenue
overall_revenue <- sum(total_monthly_revenue)
cat("Total Revenue per Month:\n")
print(data.frame(Month = months,TotalRevenue =
total_monthly_revenue))
cat("\nOverall Revenue (Jan–June):",overall_revenue,"\n\n")
avg_premium <- mean(premium_revenue)
sd_premium <- sd(premium_revenue)
cat("Average Premium Revenue:",avg_premium,"\n")
cat("Standard Deviation of Premium Revenue:",sd_premium,"\n\n")
premium_above_5000 <- months[premium_revenue > 5000]
cat("Months where Premium Revenue > $5000:\n")
print(premium_above_5000)
cat("\n")
growth_basic <- basic_revenue[6] - basic_revenue[1]
cat("Basic Revenue Growth (January to June):",growth_basic,"\n")
```

OUTPUT:

```
Basic Revenue: 2800 3050 3350 3600 3950 4250
Premium Revenue: 4800 5100 5350 5600 5900 6250

Total Revenue per Month:
     Month TotalRevenue
1  January        7600
2 February        8150
3    March        8700
4    April        9200
5      May        9850
6     June       10500

Overall Revenue (Jan–June): 54000

Average Premium Revenue: 5500
Standard Deviation of Premium Revenue: 530.0943

Months where Premium Revenue > $5000:
[1] "February" "March"    "April"    "May"      "June"

Basic Revenue Growth (January to June): 1450
```

III Matrix Operations in R

"TechStore" Quarterly Sales Performance

A technology store records quarterly sales of **Laptops, Mobiles, and Accessories** across **three branches**.

Using Matrices in R, do the following:

a. Create a matrix representing sales data where rows correspond to branches and columns represent product categories.
b. Calculate the total sales for each branch and each product.
c. Find the branch with the highest total sales.
d. Compute the average sales for each product category.
e. Access and display sales of **Mobiles** in the **second branch**.

PROGRAM:

```
branches <- c("Branch1", "Branch2", "Branch3")
products <- c("Laptops", "Mobiles", "Accessories")
sales_matrix <-
matrix(c(135000,90000,35000,160000,105000,45000,120000,80000,30000),
nrow = 3,byrow = TRUE,dimnames = list(branches, products))
cat("Sales Matrix:\n")
print(sales_matrix)
cat("\n")
branch_total <- rowSums(sales_matrix)
product_total <- colSums(sales_matrix)
cat("Total Sales per Branch:\n")
print(branch_total)
cat("\n")
cat("Total Sales per Product:\n")
print(product_total)
cat("\n")
highest_branch <- names(which.max(branch_total))
cat("Branch with Highest Total Sales:", highest_branch, "\n\n")
average_product_sales <- colMeans(sales_matrix)
cat("Average Sales per Product:\n")
print(average_product_sales)
cat("\n")
mobile_branch2 <- sales_matrix[2, "Mobiles"]
```

OUTPUT:

```
Sales Matrix:
        Laptops Mobiles Accessories
Branch1  135000   90000       35000
Branch2  160000  105000       45000
Branch3  120000   80000       30000

Total Sales per Branch:
Branch1 Branch2 Branch3
 260000  310000  230000

Total Sales per Product:
   Laptops     Mobiles Accessories
    415000      275000      110000

Branch with Highest Total Sales: Branch2

Average Sales per Product:
   Laptops     Mobiles Accessories
  138333.33    91666.67    36666.67
```

IV Matrix Operations in R

Student Marks Analysis

A class has marks in **Mathematics, Physics, and Computer Science** for **five students**.

Using Matrices in R, do the following:

a. Create a matrix to store student marks.
b. Calculate the total and average marks for each student.
c. Find the subject-wise highest and lowest marks.

d. Identify students who scored more than **75** in all subjects.
e. Replace marks below **40** with **"Fail"** indication (using logical conditions).

PROGRAM:

```
marks <- matrix(
  c(
    82, 78, 85,
    68, 72, 70,
    91, 88, 94,
    39, 45, 42,
    76, 80, 79
  ),
  nrow = 5,
  byrow = TRUE
)
rownames(marks) <- c("Student1", "Student2", "Student3", "Student4",
"Student5")
colnames(marks) <- c("Mathematics", "Physics", "Computer_Science")
cat("Student Marks Matrix:\n")
print(marks)
total_marks <- rowSums(marks)
average_marks <- rowMeans(marks)
cat("\nTotal Marks of Each Student:\n")
print(total_marks)
cat("\nAverage Marks of Each Student:\n")
print(average_marks)
highest_marks <- apply(marks, 2, max)
lowest_marks <- apply(marks, 2, min)
cat("\nHighest Marks Subject-wise:\n")
print(highest_marks)
cat("\nLowest Marks Subject-wise:\n")
```

```
print(lowest_marks)
students_above_75 <- apply(marks, 1, function(x) all(x > 75))
cat("\nStudents scoring more than 75 in all subjects:\n")
print(students_above_75)
result <- ifelse(marks < 40, "Fail", marks)
cat("\nMarks with Fail Indication:\n")
print(result)
```

OUTPUT:

```
Student Marks Matrix:
        Mathematics Physics Computer_Science
Student1          82      78             85
Student2          68      72             70
Student3          91      88             94
Student4          39      45             42
Student5          76      80             79

Total Marks of Each Student:
Student1 Student2 Student3 Student4 Student5
     245      210      273      126      235

Average Marks of Each Student:
Student1 Student2 Student3 Student4 Student5
81.66667 70.00000 91.00000 42.00000 78.33333

Highest Marks Subject-wise:
    Mathematics         Physics Computer_Science
             91              88             94

Lowest Marks Subject-wise:
    Mathematics         Physics Computer_Science
             39              45             42
```

```
Students scoring more than 75 in all subjects:
Student1 Student2 Student3 Student4 Student5
   TRUE    FALSE     TRUE    FALSE     TRUE

Marks with Fail Indication:
        Mathematics Physics Computer_Science
Student1 "82"       "78"    "85"
Student2 "68"       "72"    "70"
Student3 "91"       "88"    "94"
Student4 "Fail"     "45"    "42"
Student5 "76"       "80"    "79"
```

V Array Operations in R

Hospital Patient Statistics

A hospital records the number of patients across **3 departments**, **4 weeks**, and **2 shifts (Day/Night)**.

Using Arrays in R, do the following:

a. Create a 3-dimensional array to store patient counts.
b. Calculate the total number of patients per department.
c. Find the average number of patients per week.
d. Extract patient data for the **Night shift** only.
e. Identify the week with the highest patient count.

PROGRAM:

```
patients <- array(
  c(
    45, 50, 48, 55,
```

```
    38, 42, 40, 44,
    30, 35, 33, 36,
    25, 28, 30, 32,
    20, 22, 24, 26,
    15, 18, 20, 22
  ),
  dim = c(3, 4, 2)
)
dimnames(patients) <- list(
  Department = c("Emergency", "ICU", "OPD"),
  Week = c("Week1", "Week2", "Week3", "Week4"),
  Shift = c("Day", "Night")
)
cat("Patient Count Array:\n")
print(patients)
total_per_department <- apply(patients, 1, sum)
cat("\nTotal Patients per Department:\n")
print(total_per_department)
average_per_week <- apply(patients, 2, mean)
cat("\nAverage Patients per Week:\n")
print(average_per_week)
night_shift_data <- patients[, , "Night"]
cat("\nNight Shift Patient Data:\n")
print(night_shift_data)
total_per_week <- apply(patients, 2, sum)
highest_week <- names(which.max(total_per_week))
cat("\nWeek with Highest Patient Count:\n")
print(highest_week)
```

OUTPUT:

Patient Count Array:
, , Shift = Day

```
          Week
Department  Week1 Week2 Week3 Week4
  Emergency    45    55    40    35
  ICU          50    38    44    33
  OPD          48    42    30    36

, , Shift = Night

          Week
Department  Week1 Week2 Week3 Week4
  Emergency    25    32    24    18
  ICU          28    20    26    20
  OPD          30    22    15    22


Total Patients per Department:
Emergency       ICU       OPD
      274       259       245

Average Patients per Week:
   Week1    Week2    Week3    Week4
37.66667 34.83333 29.83333 27.33333

Night Shift Patient Data:
          Week
Department  Week1 Week2 Week3 Week4
  Emergency    25    32    24    18
  ICU          28    20    26    20
  OPD          30    22    15    22

Week with Highest Patient Count:
[1] "Week1"
```

VI Array Operations in R

Weather Data Analysis

A meteorological department collects temperature data for **3 cities**, **7 days**, and **2 time periods (Morning/Evening)**.

Using Arrays in R, do the following:

a. Create an array to store the temperature data.
b. Calculate the average temperature for each city.
c. Determine the day with the highest recorded temperature.
d. Extract temperature data for a specific city across all days.
e. Compare morning and evening temperatures using array slicing.

PROGRAM:

```
temperature <- array(
  c(

    23, 24, 25, 26, 27, 28, 29,
    18, 19, 20, 21, 22, 23, 24,
    20, 21, 22, 23, 24, 25, 26,



    27, 28, 29, 30, 31, 32, 33,
    22, 23, 24, 25, 26, 27, 28,
    24, 25, 26, 27, 28, 29, 30
  ),
  dim = c(3, 7, 2)
)

dimnames(temperature) <- list(
City = c("Chennai", "Delhi", "Mumbai"),
Day = paste0("Day", 1:7),
```

```
Time = c("Morning", "Evening")
)
cat("Temperature Data Array:\n")
print(temperature)
average_city_temp <- apply(temperature, 1, mean)
cat("\nAverage Temperature for Each City:\n")
print(average_city_temp)
max_temp_per_day <- apply(temperature, 2, max)
highest_temp_day <- names(which.max(max_temp_per_day))
cat("\nDay with Highest Recorded Temperature:\n")
print(highest_temp_day)
delhi_data <- temperature["Delhi", , ]
cat("\nTemperature Data for Delhi (All Days):\n")
print(delhi_data)
morning_temp <- temperature[, , "Morning"]
evening_temp <- temperature[, , "Evening"]
cat("\nMorning Temperatures:\n")
print(morning_temp)
cat("\nEvening Temperatures:\n")
print(evening_temp)
```

OUTPUT:

```
Temperature Data Array:
, , Time = Morning

        Day
City      Day1 Day2 Day3 Day4 Day5 Day6 Day7
  Chennai   23   26   29   20   23   21   24
  Delhi     24   27   18   21   24   22   25
  Mumbai    25   28   19   22   20   23   26
```

```
, , Time = Evening

        Day
City      Day1 Day2 Day3 Day4 Day5 Day6 Day7
  Chennai   27   30   33   24   27   25   28
  Delhi     28   31   22   25   28   26   29
  Mumbai    29   32   23   26   24   27   30


Average Temperature for Each City:
 Chennai    Delhi   Mumbai
25.71429 25.00000 25.28571

Day with Highest Recorded Temperature:
[1] "Day3"

Temperature Data for Delhi (All Days):
      Time
Day    Morning Evening
  Day1      24      28
  Day2      27      31
  Day3      18      22
  Day4      21      25
  Day5      24      28
  Day6      22      26
  Day7      25      29

Morning Temperatures:
        Day
City      Day1 Day2 Day3 Day4 Day5 Day6 Day7
  Chennai   23   26   29   20   23   21   24
  Delhi     24   27   18   21   24   22   25
```

Mumbai	25	28	19	22	20	23	26

Evening Temperatures:

City	Day1	Day2	Day3	Day4	Day5	Day6	Day7
Chennai	27	30	33	24	27	25	28
Delhi	28	31	22	25	28	26	29
Mumbai	29	32	23	26	24	27	30